CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

Support Vector Machines

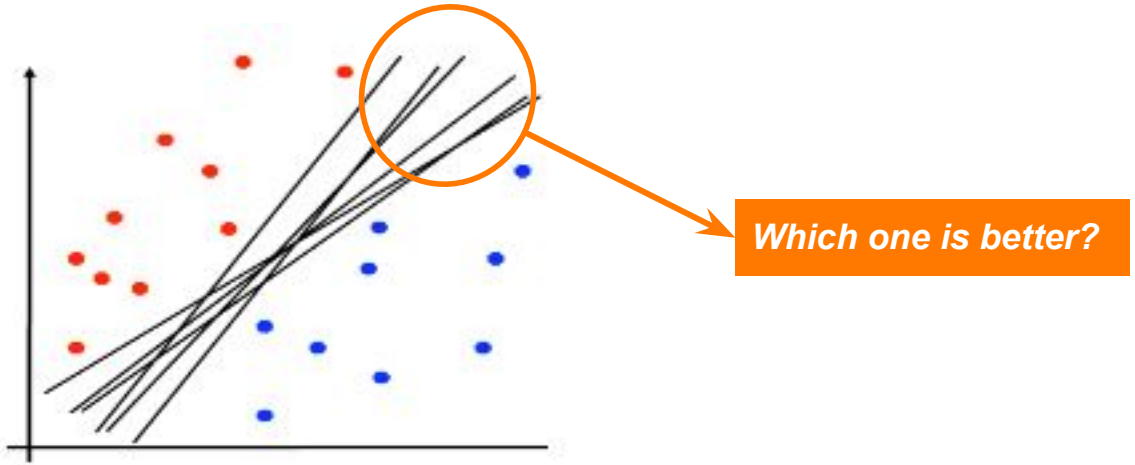
Chapter 5

- Linear SVM Classification
  - Hard vs. Soft Margin Classification
- Nonlinear SVM Classification
  - Polynomial Kernel
  - Gaussian RBF Kernel
    - Similarity Features
- SVM Regression

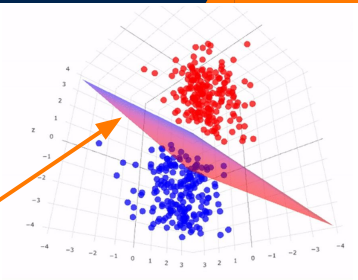
Support Vector Machines (SVMs)

What is it?

- A supervised machine learning methods proposed by Vladimir Vapnik and colleagues in the 1990s
  - i.e. use labeled datasets (i.e., classification task) to train the algorithms
  - Capable of performing both linear/nonlinear **classification** and **regression**

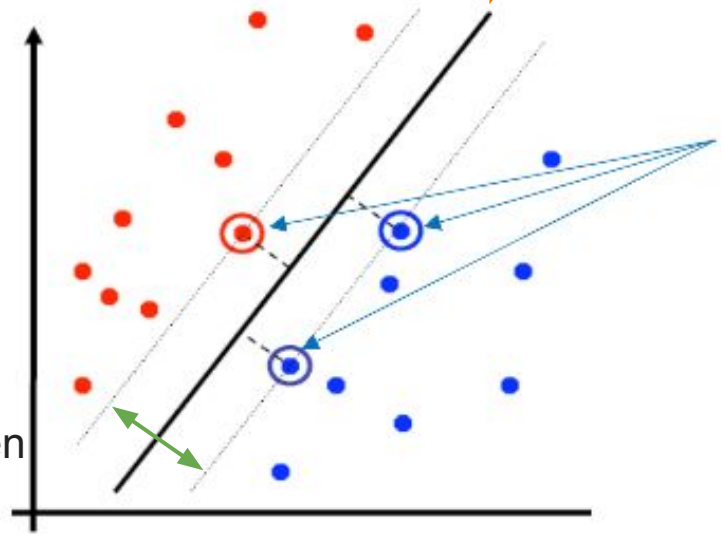


Linear SVMs Classification



Basic Idea

- Classifies data by finding an **optimal line** (or **hyperplane**) that maximizes the distance between each class in an N-dimensional space



SVM is sometimes called "Maximum Margin Classifier"

Margin

- Width of separation between the support vectors

Support Vectors

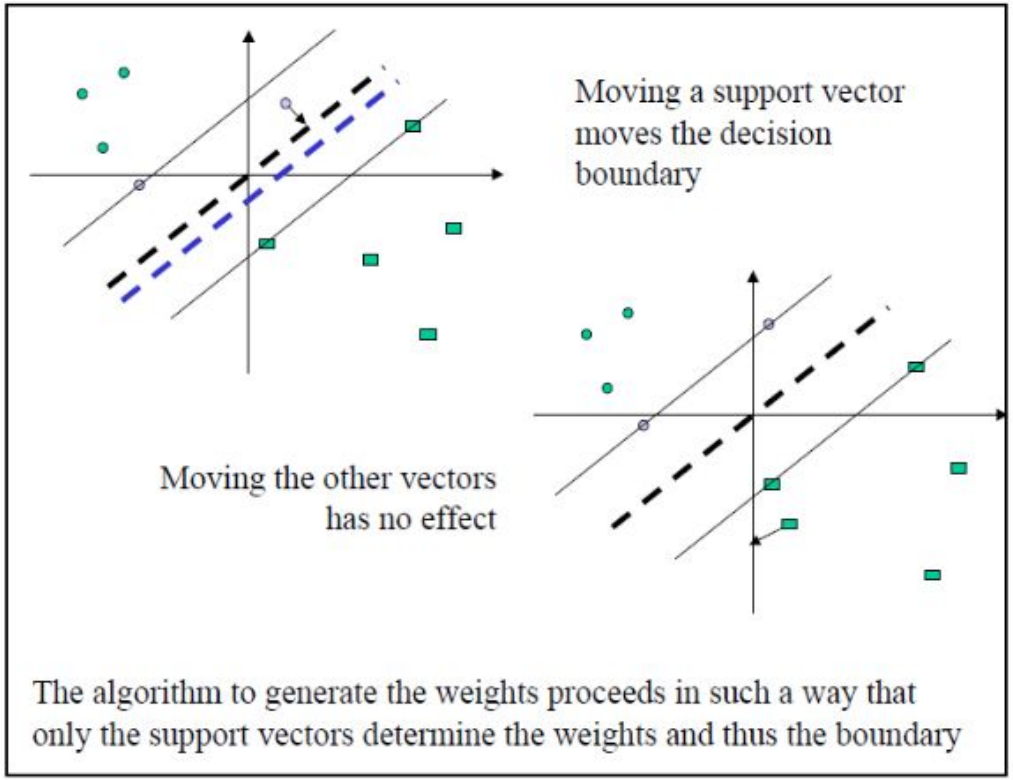
- Data points **closest to the hyperplane**
- They have direct bearing on the optimal location of the decision surface and are the most difficult to classify

Linear SVMs Classification

For classification, we only need the support vectors

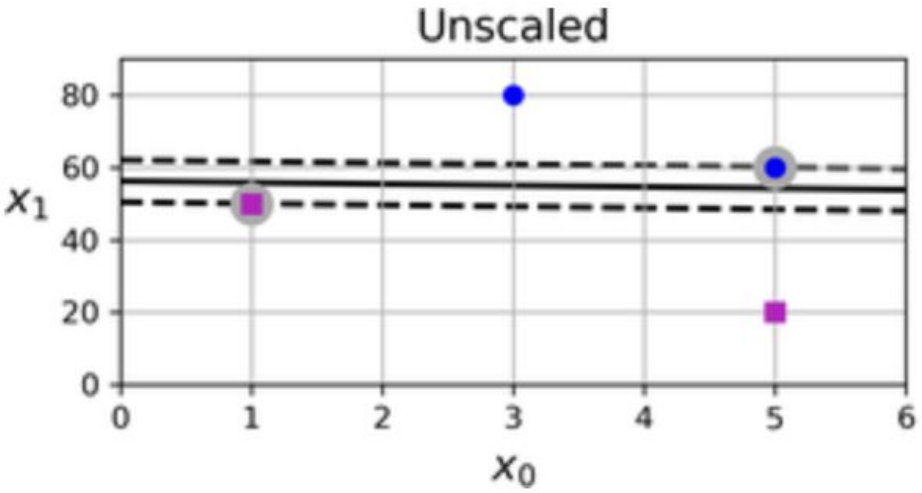
```
from sklearn.svm import SVC

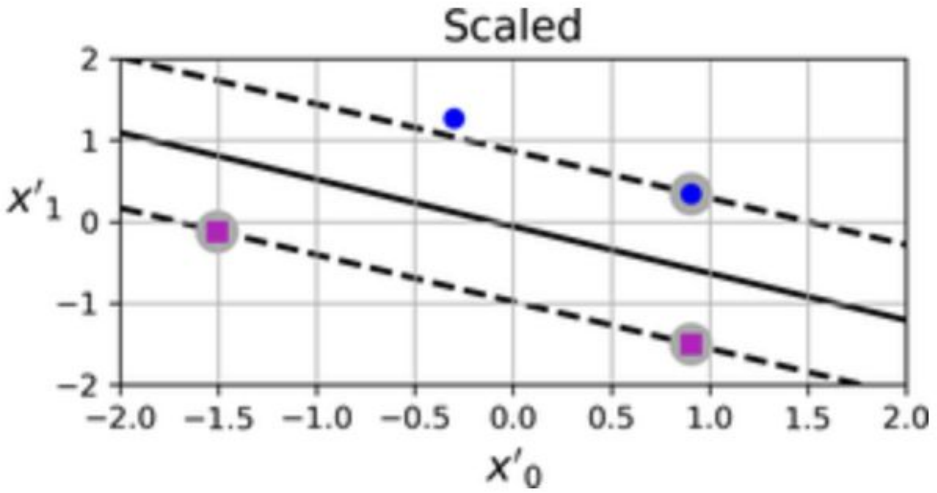
svm = SVC(kernel="linear", C=1e100)
svm.fit(X, y)
w = svm.coef_[0]
b = svm.intercept_[0]
svm.support_vectors_
```



SVM in Practice - *Feature Scaling Problem*

- SVMs are sensitive to the feature scales



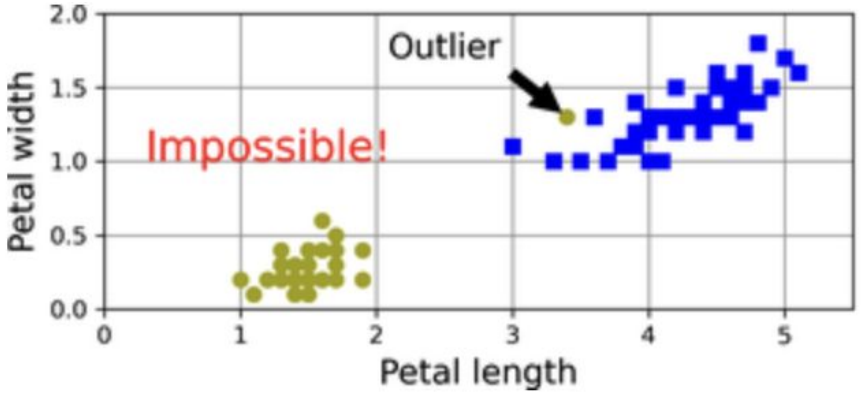


Hard Margin Classification

- When two classes can clearly be separated easily with a straight line
  - i.e., Linearly separable
- Strictly impose that ALL instances must be off the street and on the correct side

Problems with hard margin…

- NOT realistic to assume that the data are perfectly separable by the hard margin while all the constraints satisfied
- ***Sensitivity to outliers!!***



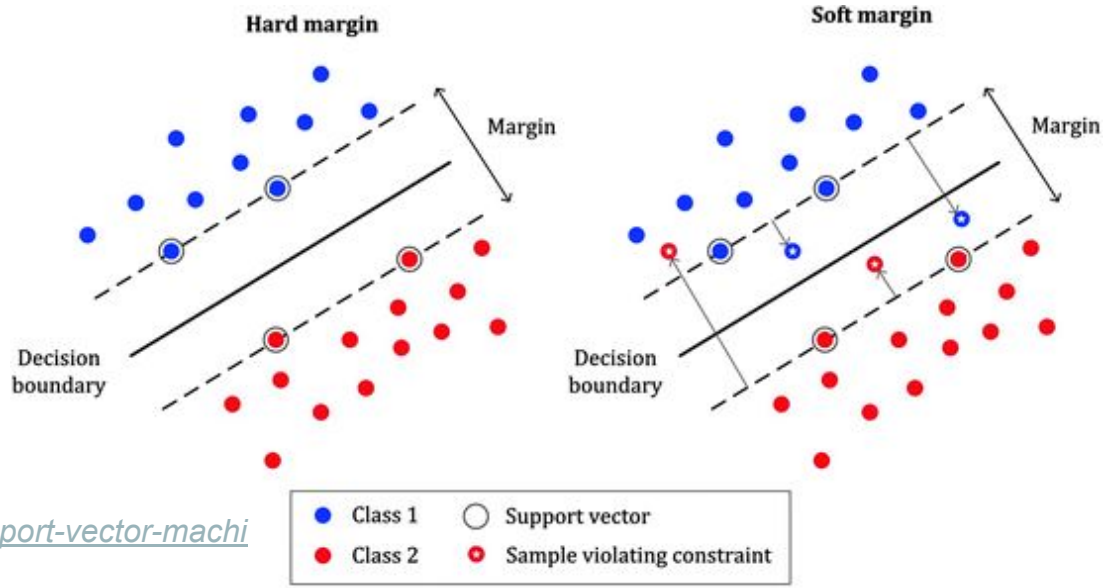
Soft Margin Classification

A more flexible model…

- We still want to place a hyperplane with large margin to minimize the number of violations

The objective is to find a ***good balance*** between keeping the street as large as possible and limiting the **margin violations**

- i.e., instances that end up in the middle of the street or even on the wrong side



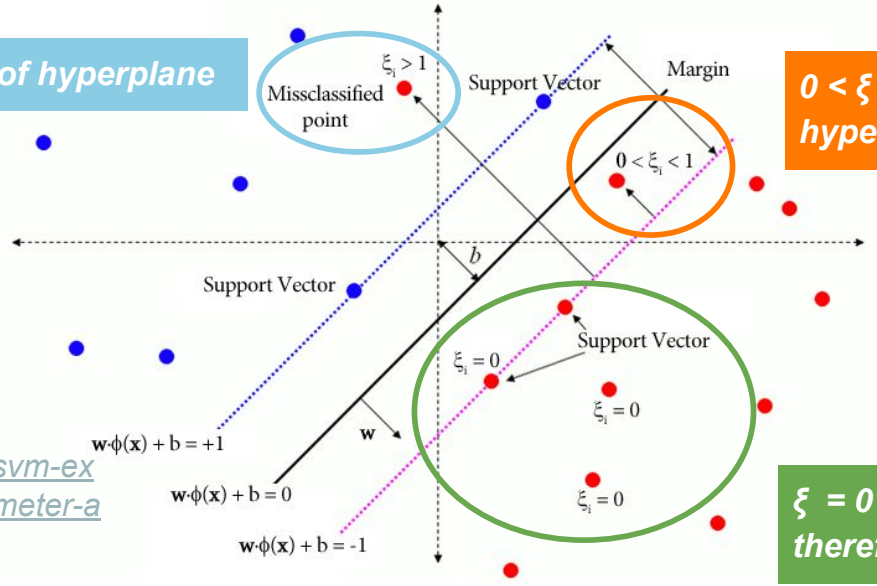
Reference:
https://ankitnitjsr13.medium.com/math-behind-svm-support-vector-machine-864e58977fdb

Soft Margin Classification

$$y(w^T x + w_0) \geq 1 - \xi$$

- Add **slack variables** ξ (for each *x*)
  - to allow misclassification (violation of constraints) of difficult or noisy examples
    - i.e., quantify amount of misclassification

ξ > 1 wrong side of hyperplane

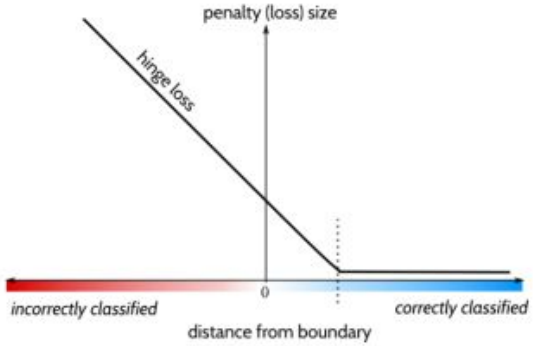


0 < ξ < 1 still on correct side of hyperplane but within the margin

ξ = 0 correct side of the margin, therefore NO violation

Reference:
https://pub.aimind.so/soft-margin-svm-exploring-slack-variables-the-c-parameter-and-flexibility-1555f4834ecc

Soft Margin Classification - *Hyperparameter C*



Idea

- For each data point, if margin ≥ 1, don't care
  - but if <1, pay a *linear penalty, **C***

Soft margin objective function

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi^{(i)}$$

such that

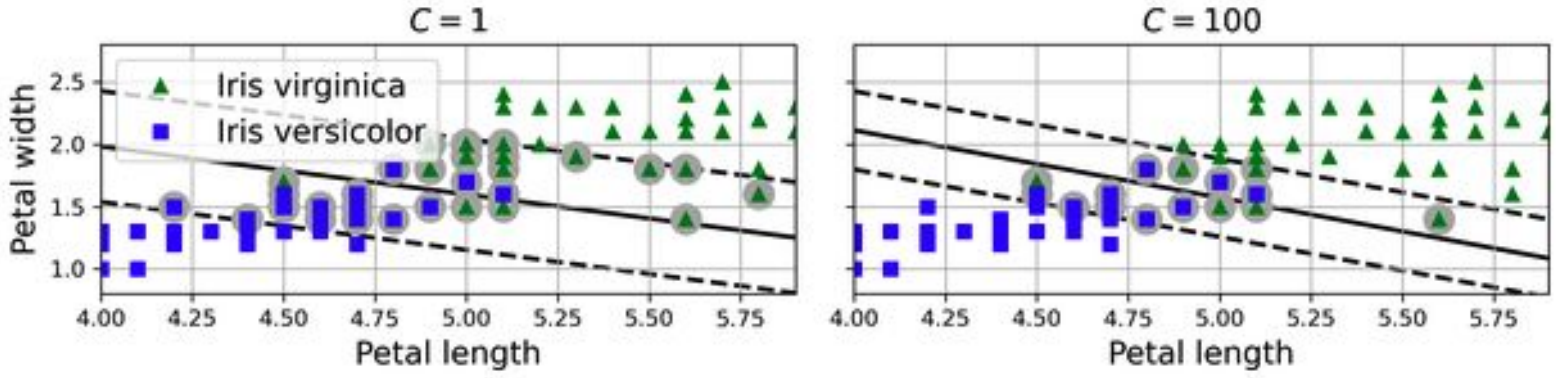
$$y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 - \xi^{(i)}$$
$$\forall_i \xi^{(i)} \geq 0$$

C is a **regularization parameter** to control the penalty for misclassification errors

- Increasing *C* -> decreases the tolerance for breaking the margin
- $C \Rightarrow \infty$ have to separate the data (hard margin)
  - $\xi = 0$ for every data point
- *C* = 0: ignore the data entirely!

Soft Margin Classification - *Hyperparameter C*

Choose the penalty $C > 0$ using cross-validation



More regularization ***Less regularization***

Note:

- Larger C: fewer violations allowed (most training instances must be correctly classified)
  - More likely to ***overfit***

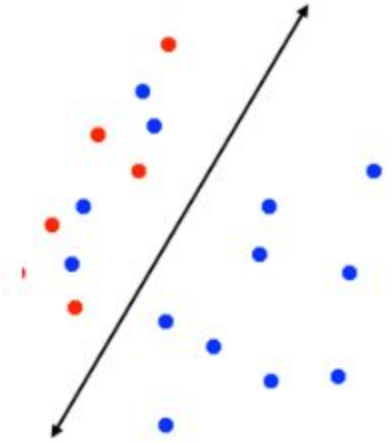
Dealing with *Imbalanced Data*

Classifying imbalanced data

- We want errors from the minority and majority classes to have a more balanced influence on the objective.
  - Otherwise, the majority class can dominate training
- A slight modification to the SVM objective does the trick: $N = N_+ + N_-$

$$\min_{\mathbf{w},\, w_0} \; ||w||_2^2 + \frac{CN}{2N_+} \sum_{j:y_j=+1} \xi_j + \frac{CN}{2N_-} \sum_{j:y_j=-1} \xi_j$$

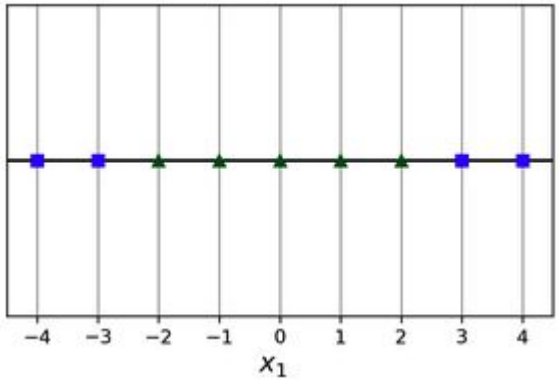
Class-specific weighting of the slack variables



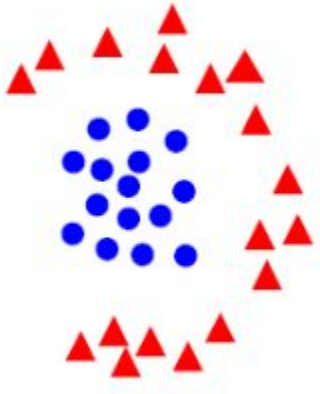
What If the Data Is Nonlinear?

Linear Separation Is NOT Always Possible

- SVM is a linear classifier using a hyperplane
  - Hyperplane is a linear subspace of a vector space



Or

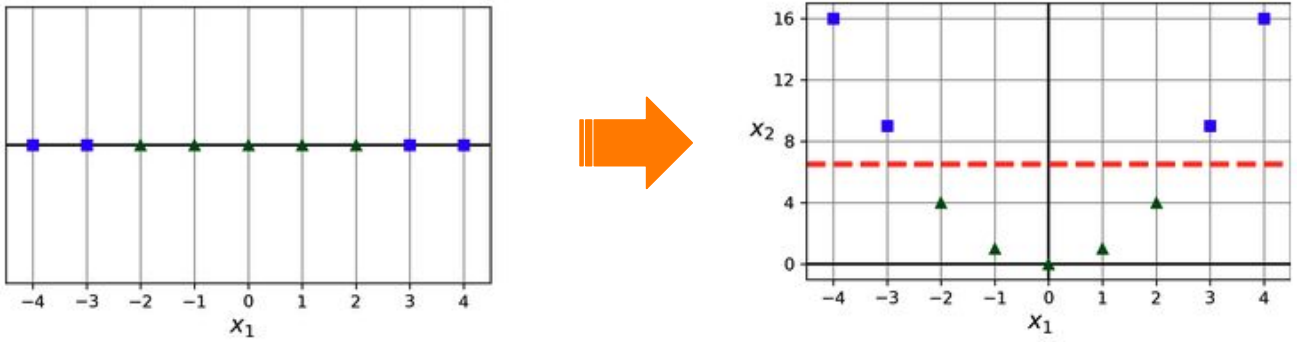


How can SVM classify data that is not linearly separable?

Nonlinear SVM Classification - *Polynomial Features*

*Mapping Data to **High Dimension***

- Points that were linearly non-separable on a line (1-D) become linearly separable in 2-D (provided an appropriate transformation was applied to the points: $x_2 = x_1^2$)



Rationale

Data that is linearly non-separable in low dimensions may become linearly separable in high dimensions (provided sensible features are chosen)

Remember?

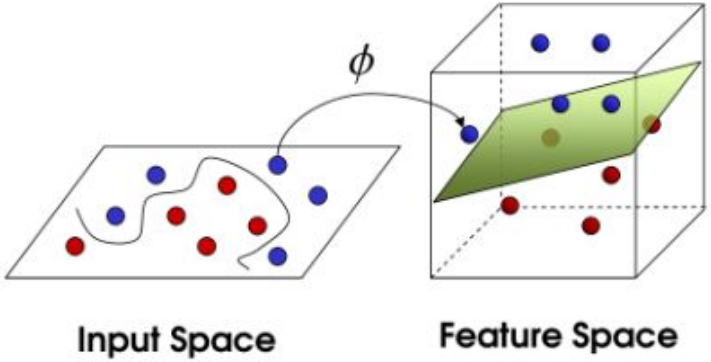
$\theta_0 + \theta_1 x \Rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$

Nonlinear SVM Classification - *Polynomial Features*

Problem?

Let the original data be m-dimensional

- Transforming to a polynomial space of degree *d* will result in how many features?



Example

- Degree 1: (x_1 , x_2)
- Degree 2: $(1, x_1, x_2 , x_1^2, x_2^2, x_1x_2)$
- Degree 3: $(1, x_1, x_2 , x_1^2, x_2^2, x_1x_2, x_1^3, x_2^3, x_1^2x_2 , x_1x_2^2)$

combinatorial explosion!!!

Kernel Trick

Lagrange Equation:

$$\underset{\alpha}{\text{minimize}} \; \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)\mathsf{T}}\mathbf{x}^{(j)}$$

Basic Idea

- We only need dot products in the transformed space

$$\Phi(x)^T \cdot \Phi(z) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)\begin{pmatrix} z_1^2 \\ z_2^2 \\ \sqrt{2}z_1z_2 \end{pmatrix} = x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2$$
$$= (x_1x_2 + z_1z_2)^2 = (x^Tz)^2$$

Takes lots of computation to calculate each in the feature space

Kernel Trick claims…

- Computing *K(x,z)* in the original input space <u>without a feature transformation</u>
- A ***computational "trick"*** to compute dot products in higher dimensional feature spaces

Support Vector Machines

Chapter 5

- Linear SVM Classification
  - Hard vs. Soft Margin Classification
- Nonlinear SVM Classification
  - **Polynomial Kernel**
  - **Gaussian RBF Kernel**
    - **Similarity Features**
- **SVM Regression**

Kernel Trick - Simple Example

Consider the following feature transformation of a 2-D vector to a 3-D vector:

$$\Phi(x_1, x_2) = \left(x_1^2, x_2^2, \sqrt{2}x_1x_2\right)$$

How do we apply this transformation to the two vectors?

- $x = (2, 3)$
- $y = (1, 4)$

$$\phi(x) = \phi(2,3) = (2^2, 3^2, \sqrt{2} * 2 * 3) = (4, 9, 6\sqrt{2})$$

$$\phi(y) = \phi(1,4) = (1^2, 4^2, \sqrt{2} * 1 * 4) = (1, 16, 4\sqrt{2})$$

Calculate the dot product between two transformed 3-D vectors

$$\begin{aligned}\phi(x)^T \cdot \phi(y) &= (4, 9, 6\sqrt{2})^T \cdot (1, 16, 4\sqrt{2}) \\ &= 4 * 1 + 9 * 16 + (6\sqrt{2}) * (4\sqrt{2}) \\ &= 4 + 144 + 48 \\ &= 196\end{aligned}$$

Calculate the square of the dot product of the original vectors

$$\begin{aligned}(x^T \cdot y)^2 &= \left[(2, 3)^T \cdot (1, 4)\right]^2 \\ &= (2 * 1 + 3 * 4)^2 \\ &= 14^2 \\ &= 196\end{aligned}$$

Kernel Trick - Practice!!!

Consider the following feature transformation of a 3-D vector to a 6-D vector:

$$\phi(x_1, x_2, x_3) = (x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3)$$

Can you proof that computing the dot product in the transformed feature space gives the same result as applying a degree-2 polynomial kernel directly in the original space?

- $x = (1, 2, -1)$
- $y = (0, -2, 1)$

$$(x^T \cdot y)^2$$

Polynomial Kernels

$$K(x, z) = (x \cdot z)^d$$

Polynomials of degree exactly degree *d*

- *d* = *1* (linear kernel)

$$\Phi(x) \cdot \Phi(z) = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \cdot \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = x_1 z_1 + x_2 z_2 = x \cdot z$$

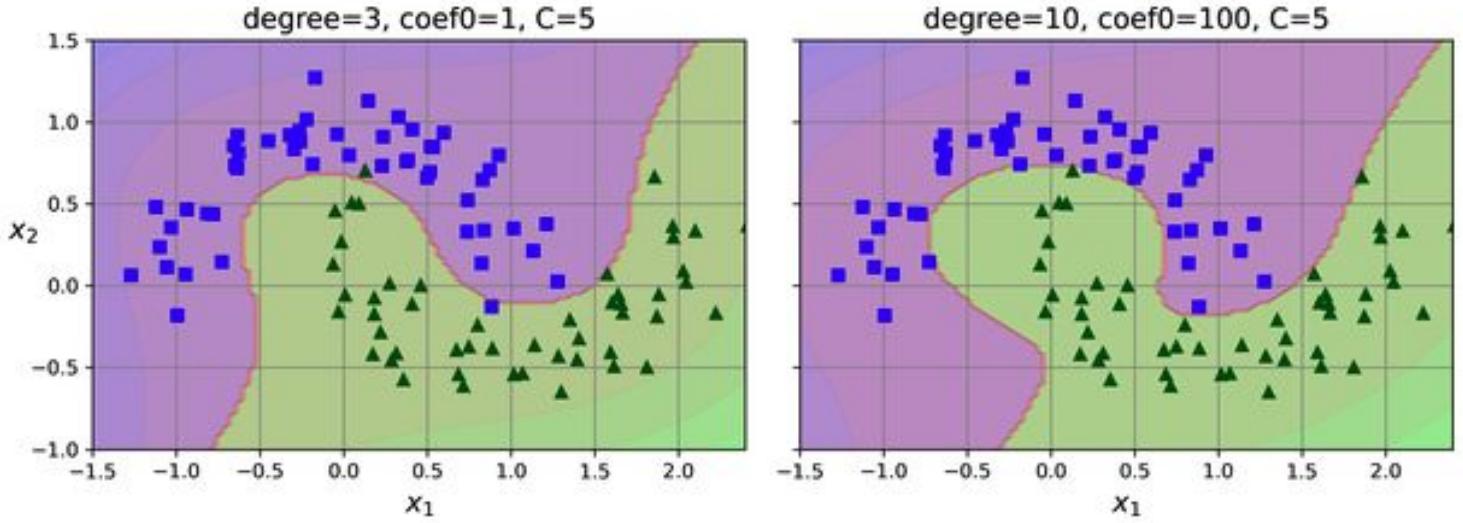
- *d* = *2* (quadratic kernel)

$$\Phi(x) \cdot \Phi(z) = \begin{pmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{pmatrix} \cdot \begin{pmatrix} z_1^2 \\ \sqrt{2} z_1 z_2 \\ z_2^2 \end{pmatrix} = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2$$

$$= (x_1 z_1 + x_2 z_2)^2 = (x \cdot z)^2$$

- For any *d*, $\Phi(x) \cdot \Phi(z) = (x \cdot z)^d$ (proof is skipped)

Polynomial Kernels



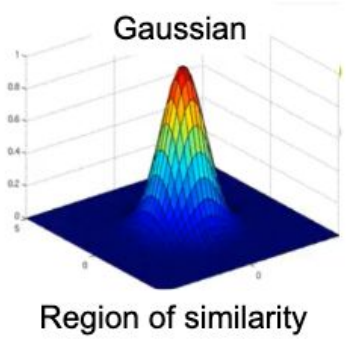

```
from sklearn.svm import SVC

svm = SVC(kernel="poly", degree=3, coef0=1, C=5)
svm.fit(X, y)
```

```
from sklearn.svm import SVC

svm = SVC(kernel="poly", degree=10, coef0=100, C=5)
svm.fit(X, y)
```

Gaussian RBF Kernel - *Similarity Features*



Add features computed using a similarity function

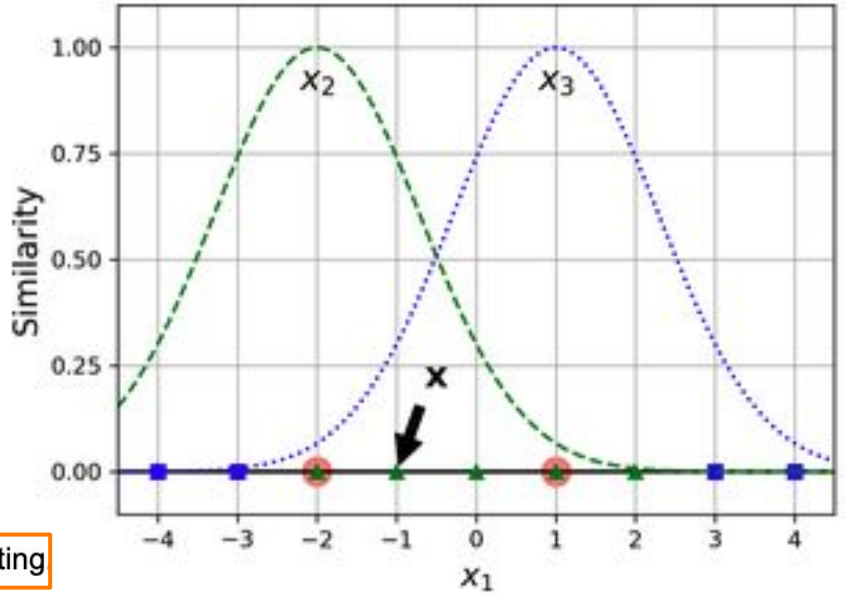
- Measures how much each instance resembles a particular **landmark**

Mathematically speaking…

- Gaussian Radial Basis Function (RBF)

$$K(x, z) = exp(-\gamma||x - z||^2)$$

$K(x, z) \approx 1$ when x, z close
$K(x, z) \approx 0$ when x, z dissimilar

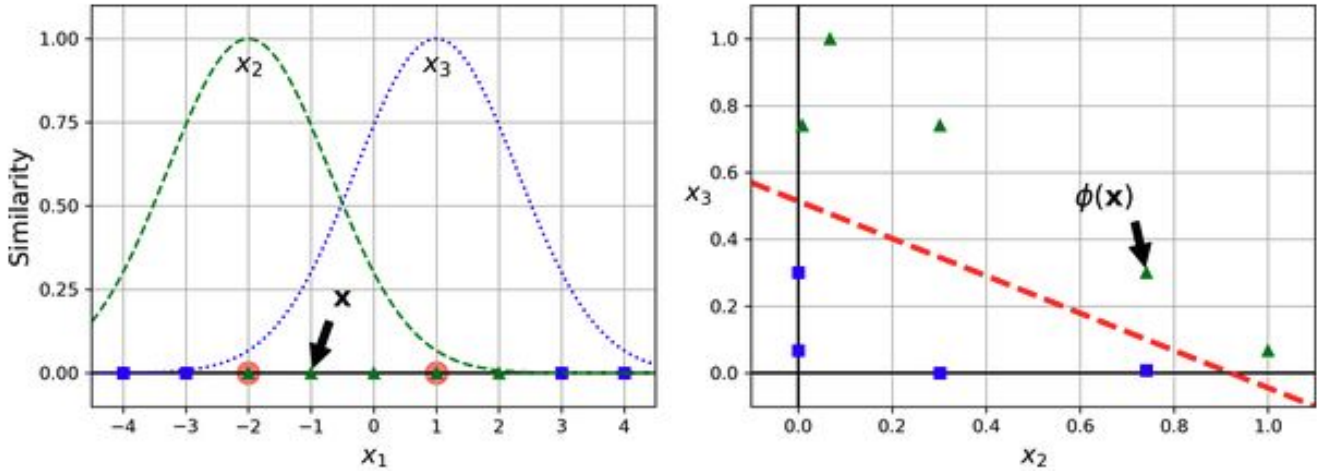


The small γ (σ is large) behaves like linear SVM, large γ may cause overfitting

Gaussian RBF Kernel - *Similarity Features*

How to select landmarks?

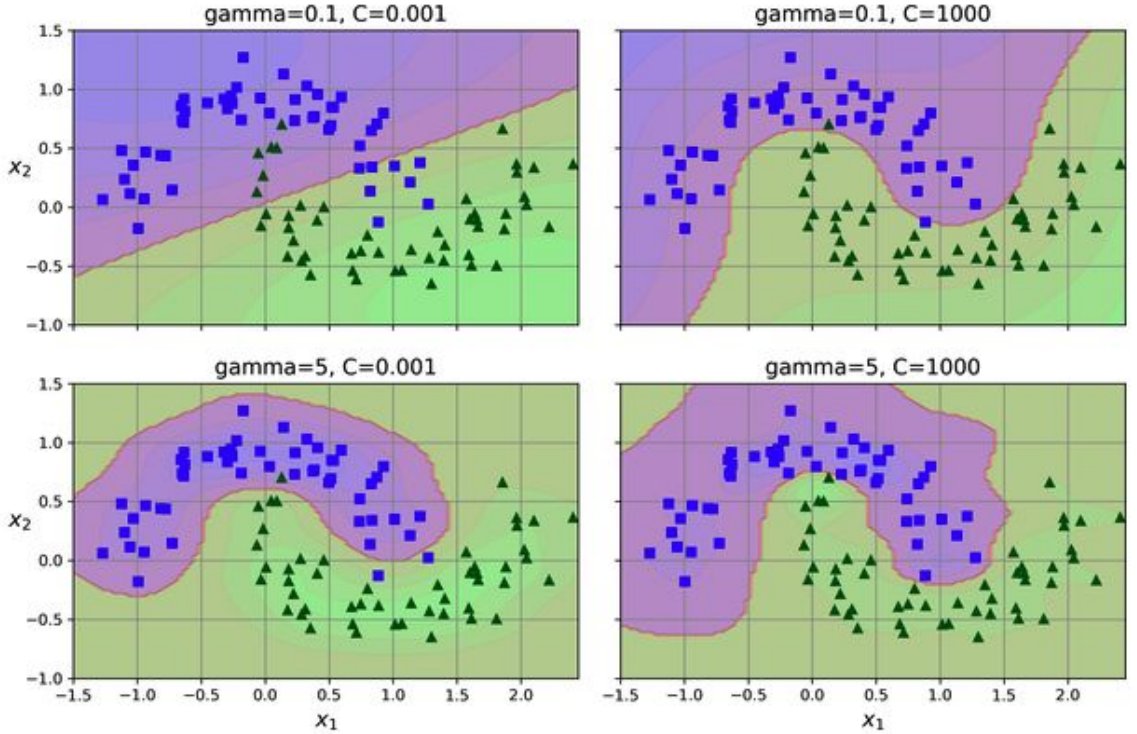
- The simplest approach is to create a landmark at the location of each and EVERY instance in the dataset



Downside:

- If your training set is very large, you end up with an equally large number of features

Gaussian RBF Kernel




```
from sklearn.svm import SVC

svm = SVC(kernel="rbf", gamma=5, C=0.001)
svm.fit(X, y)
```

Large C + **Large gamma** -> Easily *overfit*
Small C + **Small gamma** -> Easily *underfit*

Which Kernels to Use?

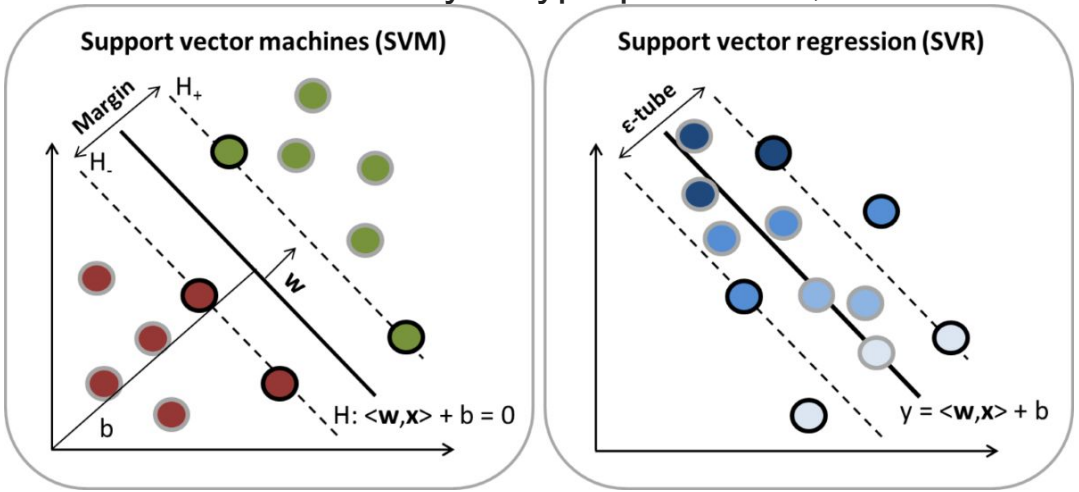
As a rule of thumb…

- Always try the linear kernel first
  - *LinearSVC* class is much faster than *SVC(kernel="linear")*, especially if the training set is very large
- If it is not too large, you should also try kernelized SVMs, starting with the *Gaussian RBF* kernel
- Then, if you have spare time and computing power, you can experiment with a few other kernels using hyperparameter search

Support Vector Regression (SVR)

Objective is tweaked

- SVM regression tries to fit as many instances as possible **inside the margin** while limiting margin violations
  - i.e., instances off the street
- The width of the street is controlled by a hyperparameter, **ε**

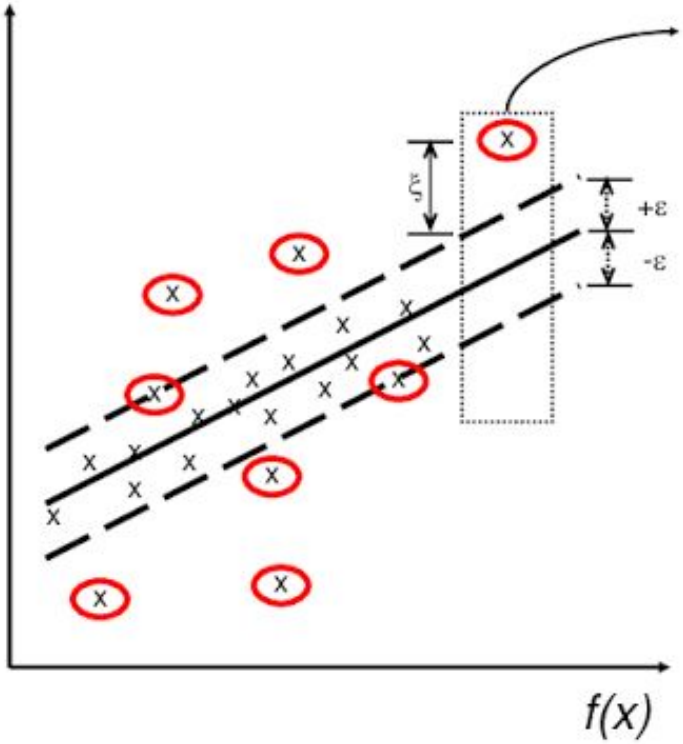


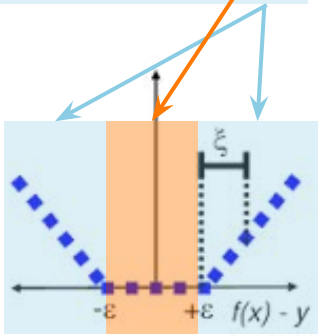
Support Vector Regression (SVR)

$$L(y, f(x)) = \begin{cases} 0 & \text{if } |y - f(x)| \leq \varepsilon \\ |y - f(x)| - \varepsilon & \text{if } |y - f(x)| > \varepsilon \end{cases}$$

Epsilon-insensitive Loss

- A regression loss function that ignores prediction errors smaller than a tolerance threshold (ε),
  - Treating them as zero loss



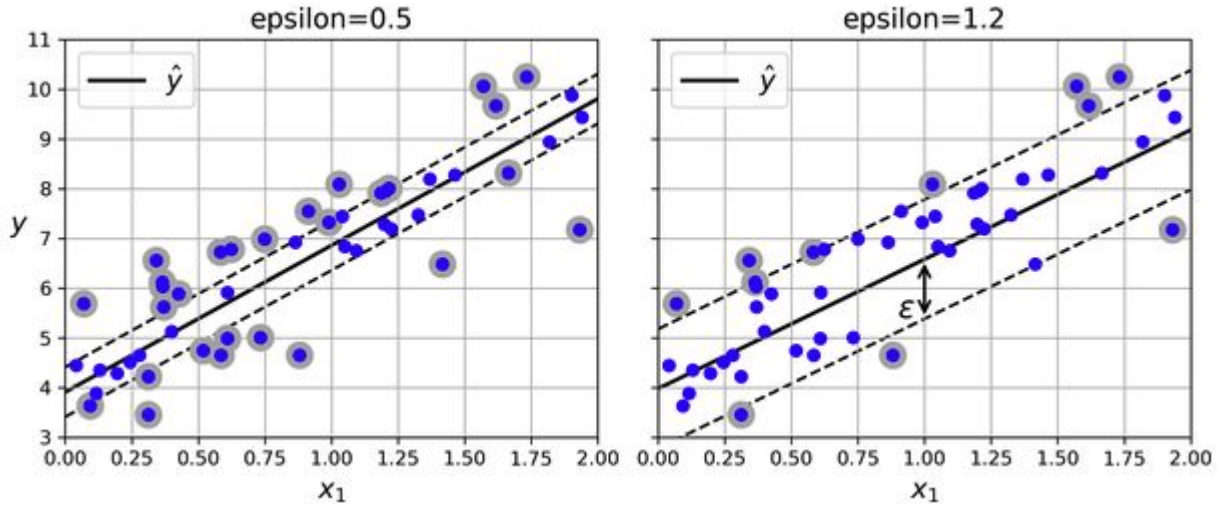


Mathematically speaking…

$$L(y, f(x)) = \max(0, |y - f(x)| - \varepsilon)$$

- Inside the "tube" where *(|y - f(x)| ≤ ε)*
  - Loss = 0
- Outside the “tube” where *(|y - f(x)| > ε)*
  - Loss grows linearly

What Happens When Epsilon Changes?



Smaller ε: Narrow tube

```
from sklearn.svm import LinearSVR

svr = LinearSVR(epsilon=0.5)
svr.fit(X, y)
```

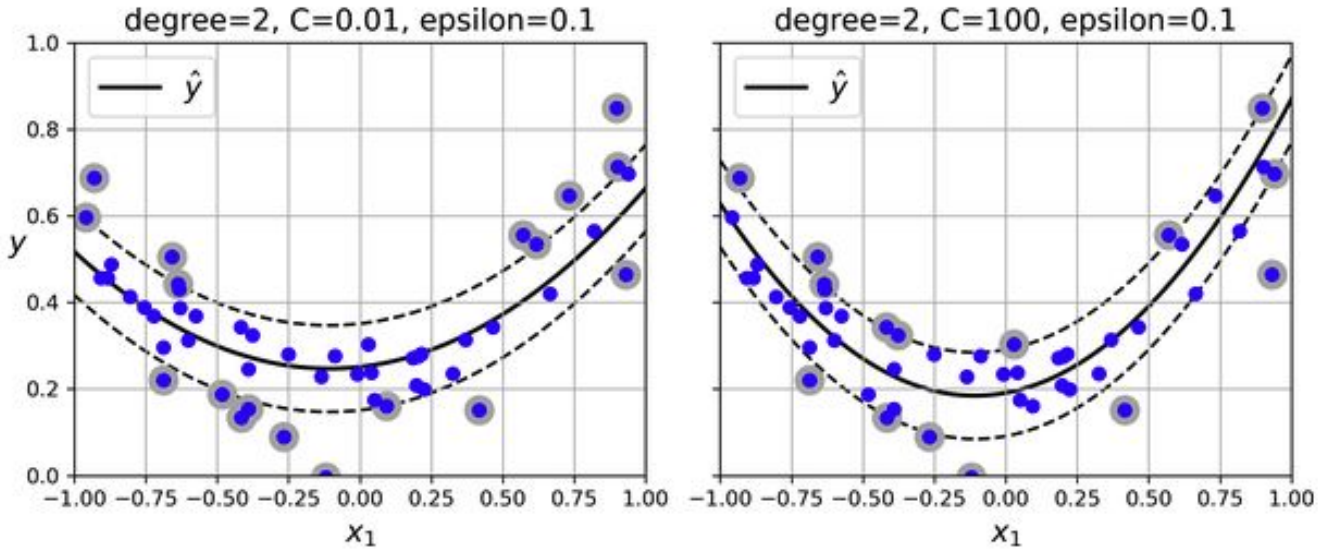
Larger ε: Wider tube

```
from sklearn.svm import LinearSVR

svr = LinearSVR(epsilon=1.2)
svr.fit(X, y)
```

Can SVR Be Nonlinear?

Yes!!!!




```
from sklearn.svm import SVR

svr = SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1)
svr.fit(X, y)
```

```
from sklearn.svm import LinearSVR

svr = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svr.fit(X, y)
```